

Prototype

1. Explicación del patrón

Prototype es un patrón de creación que permite crear nuevos objetos copiando una instancia ya existente, llamada prototipo.

En lugar de crear un objeto desde cero usando directamente su constructor, se parte de un objeto ya creado y se obtiene una copia mediante un método de clonación.

La idea principal es que el propio objeto sabe cómo copiarse a sí mismo. Por eso, el cliente no necesita conocer la clase concreta del objeto que está copiando.

Esto permite crear objetos nuevos a partir de otros ya configurados, manteniendo al cliente independiente de la forma en que esos objetos se crean, se componen o se representan.

También permite realizar acciones temporales con copias de objetos sin afectar al objeto original.

2. Cuándo se utiliza

Se utiliza el patrón Prototype cuando:

- Se quiere crear un objeto copiando otro objeto existente.
- El cliente necesita realizar cambios temporales sobre una copia sin modificar el original.
- El sistema debe ser independiente de la forma en que se crean, componen o representan los productos.
- Las clases que se deben instanciar se especifican en tiempo de ejecución.
- Una instancia de una clase puede tener uno de varios estados posibles.
- Se quiere evitar que el código cliente dependa de clases concretas.
- Se tienen objetos ya configurados y resulta más cómodo clonarlos que construirlos desde cero.
- Crear un objeto desde cero es más costoso o más complejo que copiar uno existente.
- Se quiere reducir la cantidad de subclasses que solo se diferencian en la configuración inicial de los objetos.

3. Cómo aplicarlo

Para aplicar Prototype se suelen definir estos participantes:

Prototipo

Es la interfaz o clase base que declara el método de clonación.

Responsabilidad principal:

- Definir una operación común para clonar objetos.

Operación habitual:

- `clonar()`

El cliente trabaja con esta interfaz, no con las clases concretas.

Prototipo concreto

Es una clase concreta que implementa el método `clonar()` .

Responsabilidades:

- Copiarse a sí misma.
- Crear una nueva instancia con el mismo estado que el objeto original.
- Devolver la copia creada.
- Mantener el objeto original sin modificar.

Ejemplos conceptuales:

- `PrototipoConcreto1`
- `PrototipoConcreto2`

Ejemplos del material:

- `Circulo`
- `Cuadrado`

Cliente

Es quien necesita crear una copia de un objeto.

Responsabilidades:

- Usar la interfaz `Prototipo` .
- Pedir una copia mediante `clonar()` .
- Trabajar con la copia sin conocer la clase concreta del objeto.
- Modificar la copia si lo necesita, sin alterar el original.

4. Forma práctica de aplicarlo

Pasos:

1. Identificar objetos que se crean muchas veces con estados parecidos.
2. Crear una interfaz o clase base `Prototipo` .
3. Declarar en ella el método `clonar()` .
4. Hacer que las clases concretas implementen ese método.
5. Dentro de `clonar()` , crear una nueva instancia del mismo tipo que el objeto original.
6. Copiar en la nueva instancia los datos relevantes del objeto original.
7. Devolver la copia.
8. Sustituir la creación directa de objetos por llamadas a `clonar()` .
9. Modificar únicamente la copia cuando se necesiten cambios temporales.
10. Si hay prototipos frecuentes, guardarlos en un registro o catálogo para poder clonarlos cuando se necesiten.

5. Idea clave

El cliente no debe depender de la clase concreta que quiere copiar.

El propio objeto debe proporcionar una forma de clonarse.

Así se pueden crear copias de objetos existentes sin afectar al original y sin acoplar el código cliente a clases concretas.

6. Ejemplo del material

Ejemplo de figuras

En el material aparece un ejemplo con figuras.

Existe una interfaz `Figura`, implementada por:

- `Circulo`
- `Cuadrado`

La interfaz `Figura` define operaciones como:

- `getNombre()`
- `setNombre(String n)`
- `mover(int x, int y)`
- `getPosicion()`
- `clonar()`

Cada figura concreta puede clonarse a sí misma.

La utilidad práctica del ejemplo es poder crear una copia de un círculo o de un cuadrado y modificar esa copia sin afectar al objeto original.

Por ejemplo, se quiere cambiar de sitio y de color una figura. Para no alterar la figura original, se clona primero y después se modifican los datos de la copia.

Así se pueden mostrar tanto el objeto original como el objeto copiado, comprobando que los cambios realizados sobre la copia no afectan al original.

7. Relación con Factory Method en el ejemplo

En el material se indica que también se quiere atrasar la decisión de qué clase crear.

Por eso, en el ejemplo de figuras se combina Prototype con una estructura de tipo Factory Method.

Aparecen elementos como:

- `Creador`
- `CreadorConcreto`
- `Factory_method()`

La fábrica ayuda a decidir qué figura crear o clonar, mientras que Prototype permite obtener copias de esas figuras sin modificar los objetos originales.

8. Ejemplo conceptual de uso

En vez de crear siempre objetos desde cero:

```
new Circulo()
```

o:

```
new Cuadrado()
```

se puede partir de un objeto ya existente:

```
figuraClonada = figuraOriginal.clonar()
```

Después se modifica solo la copia:

```
figuraClonada.mover(10, 20)
```

El objeto original conserva su estado inicial.

9. Ventajas prácticas

- Permite crear objetos copiando otros ya existentes.
- Evita que el cliente dependa de clases concretas.
- Permite modificar copias sin afectar al objeto original.
- Puede evitar código repetido de inicialización.
- Facilita crear objetos complejos a partir de prototipos ya configurados.
- Permite seleccionar la clase concreta en tiempo de ejecución.
- Puede reducir la necesidad de crear muchas subclases para distintas configuraciones.
- Encapsula la lógica de copia dentro del propio objeto.

10. Cuidado

Hay que tener cuidado con el tipo de copia que se realiza.

Si el objeto contiene referencias a otros objetos, puede ser necesario decidir si la copia debe ser superficial o profunda.

Una copia superficial puede compartir objetos internos con el original.

Una copia profunda crea también copias de los objetos internos.

Si el objeto es complejo o tiene referencias circulares, clonar correctamente puede ser más difícil.

11. Diferencia práctica con Factory Method

Factory Method crea objetos delegando la decisión de la clase concreta en un método fábrica o en una subclase creadora.

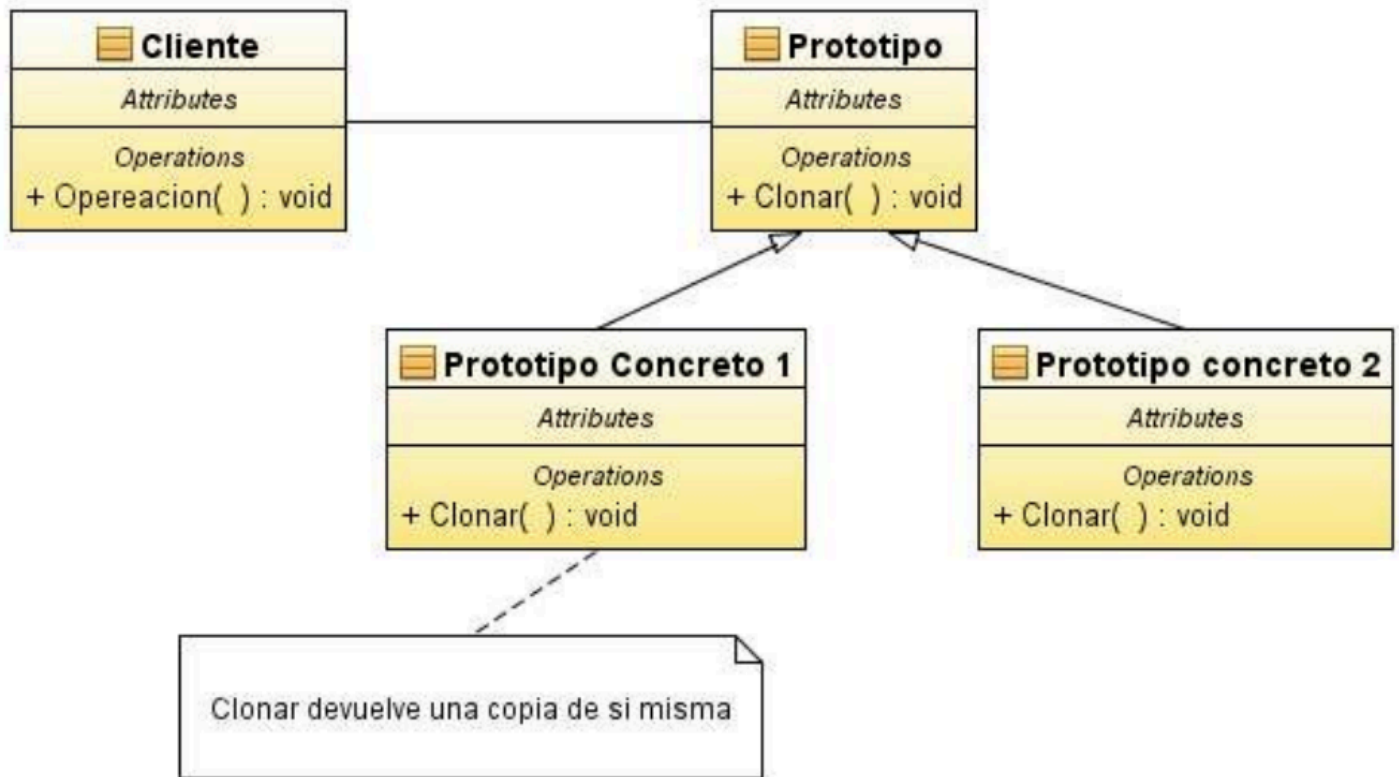
Prototype crea objetos copiando una instancia existente.

Por tanto:

- Factory Method se basa en crear un nuevo producto mediante una fábrica.
- Prototype se basa en clonar un objeto ya construido.

En el ejemplo del material pueden combinarse: Factory Method ayuda a decidir qué figura usar, y Prototype permite clonar esa figura.

12. Plantilla



13. Ejemplo

